# 📘 Building a Private Chatbot with Generative AI

## 1. Introduction

This document explains the architecture and workflow of a private chatbot built using **OpenAI's Generative AI models**. The chatbot leverages embeddings and a vector database to provide context-aware responses to user queries.

## 2. System Flow Overview

The chatbot follows a **retrieval-augmented generation (RAG)** pipeline:

1. **Source Data Ingestion**
    - Raw documents or text sources are provided.
    - Examples: PDFs, knowledge base articles, FAQs.
2. **Chunking**
    - Large documents are split into smaller, manageable text chunks.
    - This ensures embeddings capture fine-grained meaning.
3. **Embedding Generation**
    - Each chunk is converted into a numerical vector (embedding) using **OpenAI's embedding model**.
    - Embeddings represent semantic meaning.
4. **Vector Database Storage**
    - Embeddings are stored in a **vector database** (e.g., Pinecone, Weaviate, FAISS).
    - Enables efficient similarity search.
5. **User Query Processing**
    - User submits a query.
    - Query is also converted into an embedding.
6. **Similarity Search**
    - The vector database performs a similarity search between the query embedding and stored embeddings.

    - Top-ranked results are retrieved.
7. **Response Generation**
    - Retrieved context is passed to the **OpenAI LLM**.
    - The LLM generates a coherent, context-aware response.

**3. Flowchart Diagram**

Code

User Query

↓

Convert Query → Embedding

↓

Similarity Search in Vector DB

↓

Retrieve Ranked Results

↓

Pass Context → OpenAI LLM

↓

Generate Final Response

↓

Return Answer to User

**4. Example Workflow**

- **Step 1:** Upload company FAQs as source documents.
- **Step 2:** System chunks FAQs into smaller sections.
- **Step 3:** Each section is embedded and stored in Pinecone.
- **Step 4:** User asks: *"How do I reset my password?"*
- **Step 5:** Query embedding is generated.

- **Step 6:** Vector DB retrieves the most relevant FAQ chunk.
- **Step 7:** OpenAI LLM uses retrieved context to generate: *"To reset your password, go to the login page, click 'Forgot Password,' and follow the instructions sent to your email."*

**5. Key Components**

- **OpenAI Embeddings API** – Converts text into semantic vectors.
- **Vector Database (e.g., Pinecone)** – Stores and retrieves embeddings efficiently.
- **OpenAI LLM (e.g., GPT-4)** – Generates natural language responses using retrieved context.

**6. Benefits**

- **Contextual Accuracy** – Responses are grounded in stored knowledge.
- **Scalability** – Handles large document sets with efficient search.
- **Privacy** – Works with private datasets, ensuring secure knowledge retrieval.